Daily Quant Digest: 2026-11-02

Foundations Restart, day 7. Yesterday was revision. Today three new steps: when does news tell you **nothing** (independence, and why it is not the same as "cannot happen together"), how a forward becomes a **futures** contract once an exchange and a clearing house step in (margin and daily mark-to-market), and the **two pointers** technique, used to find three trades that net to zero.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 7: independence, and why independent is not the same as "cannot happen together"	Easy-Medium
Derivatives	FR Lesson 7: futures vs forwards: exchanges, clearing houses, margin, daily mark-to-market	Easy-Medium
Algorithms & DS	FR Lesson 7: two pointers: pair sum in a sorted array, then 3Sum (three trades that net to zero)	Medium
Music	Cream, "Sunshine of Your Love" (1967)	

1. Probability & Statistics: News That Tells You Nothing

> **Where we are.** Lesson 1: the sample space is a **room of possible worlds**, and a probability is a share of the room. Lesson 2: "A and B" is the overlap $A \cap B$, and "not A" is the complement. Lesson 6: news **moves the walls in**, and $P(A \mid B) = P(A \cap B)/P(B)$ is the share of the small room that A takes. Today we ask the opposite question: what if the walls move but **the share does not change**?

1.1 The central picture: the walls move, the share stays

Picture the room of possible worlds. Event A is painted red: it covers some share of the floor, say 30%. A friend gives you news B, and the walls slide in to the smaller room B.

Now look at the red paint inside the small room. Two things can happen:

- **The red share changes.** Maybe the small room is 60% red, or 5% red. Then the news B told you something about A. The events are **dependent**.
- **The red share is still exactly 30%.** The small room looks like a scaled-down copy of the big room, as far as A is concerned. The news was useless for guessing A. The events are **independent**.

That is the whole idea. **Independent means: learning B does not change your odds on A.**

In the picture	In the maths
The red paint on the floor	Event A, with share $P(A)$
The news that moves the walls	Event B
Red share inside the small room	$P(A \mid B)$
"The share did not change"	$P(A \mid B) = P(A)$: independence
"The red paint is entirely outside the small room"	$A \cap B$ is empty: A and B are **disjoint**

Look at the last row. If A and B cannot happen together, then once you hear B, the red share drops to **zero**. That is a huge change, so disjoint events are about as **dependent** as events can be. Hold on to that; it is the main trap of the day.

1.2 Count first: 1,000 traders, two different firms

Take 1,000 traders again: 400 trade FX and 600 trade rates. The event is "had a losing month".

Firm X. 300 traders lost money: 120 FX and 180 rates.

	Lost	Did not lose	Total
FX	120	280	400
Rates	180	420	600
Total	300	700	1,000

- Before any news: $P(\text{lost}) = 300/1{,}000 = 30\%$.
- News "she trades FX": the small room is the FX row. $P(\text{lost} \mid \text{FX}) = 120/400 = 30\%$.
- News "she trades rates": $180/600 = 30\%$.

The news changes nothing. At Firm X, desk and losing month are **independent**.

Firm Y is Lesson 6's firm: 160 stop-losses overall (16%), but 25% among FX traders. There the news "she trades FX" pushed the share from 16% up to 25%. **Dependent.**

And "FX" vs "rates"? Every trader is on exactly one desk, so "trades FX" and "trades rates" are disjoint. Before any news, $P(\text{rates}) = 60\%$. After the news "she trades FX", it is **0%**. Disjoint, so very dependent.

1.3 The formula, one step at a time

In words: A and B are independent when the chance of both is the chance of one times the chance of the other.

$$A \text{ and } B \text{ are independent } \iff P(A \cap B) = P(A)\,P(B).$$

Where it comes from. Start from "the share does not change", $P(A \mid B) = P(A)$. Lesson 6's multiplication rule says $P(A \cap B) = P(B) \times P(A \mid B)$. Replace $P(A \mid B)$ by $P(A)$:

$$P(A \cap B) = P(B) \times P(A).$$

Read aloud: "the probability of A and B equals the probability of A times the probability of B."

With numbers (Firm X). $P(\text{FX}) = 0.4$, $P(\text{lost}) = 0.3$, product 0.12. And the table says $P(\text{FX and lost}) = 120/1{,}000 = 0.12$. Equal, so independent.

Why use the product form? Two reasons. It is symmetric (it treats A and B the same way), and it still makes sense when $P(B) = 0$, where $P(A \mid B)$ is not defined.

Disjoint is never independent (except in a trivial case). If A and B are disjoint, $P(A \cap B) = 0$. For independence we would need $P(A)P(B) = 0$, which means one of them is impossible. FX vs rates: 0 against $0.4 \times 0.6 = 0.24$. Not equal.

Bonus: "not" keeps independence. If A and B are independent, so are A and "not B". At Firm X: FX and **did not lose** is 280, and $0.4 \times 0.7 = 0.28$. Common sense: if knowing "she lost" tells you nothing about her desk, knowing "she did not lose" cannot tell you anything either.

More than two events. Three events are (mutually) independent when every product rule holds: each pair, and also all three together, $P(A \cap B \cap C) = P(A)P(B)P(C)$. Example: three separate risk checks, each missing a bad trade 10% of the time, independently. All three miss with probability $0.1 \times 0.1 \times 0.1 = 0.001$.

1.4 A dice surprise

Roll two fair dice: 36 equally likely worlds. Let A = "first die shows 6", so $P(A) = 6/36 = 1/6$.

- B = "the sum is 7". There are 6 worlds: (1,6), (2,5), (3,4), (4,3), (5,2), (6,1). So $P(B) = 1/6$. Exactly one of them, (6,1), has a first 6. $P(A \cap B) = 1/36 = \frac{1}{6} \times \frac{1}{6}$. **Independent!**
- C = "the sum is 8". 5 worlds: (2,6), (3,5), (4,4), (5,3), (6,2). $P(C) = 5/36$. One has a first 6, so $P(A \cap C) = 1/36$. But $P(A)P(C) = 5/216 \neq 1/36$. **Dependent.**

Why is 7 special? Whatever the first die shows, there is **exactly one** second number that makes the total 7. So the total being 7 says nothing about the first die. For a total of 8, a first die of 1 makes it impossible, which is information. Independence is about the

numbers, not about whether the events “feel connected”. Here the sum obviously involves the first die, and it is still independent of it.

1.5 Common mistakes

- **Mixing up independent and disjoint.** Disjoint means “the news kills A”. Independent means “the news leaves A alone”. Opposite ideas.
- **Assuming independence because it is convenient.** The three risk checks give 0.001 only if they fail separately. If all three read the same data feed and the feed breaks, they all miss together, and the true risk can be far higher. Before 2008, many mortgage models treated defaults in different US regions as close to independent. When house prices fell nationwide, they defaulted together.
- **Checking only the pairs.** For three or more events, every pair can be independent while the three together are not. The interview question shows this.

1.6 Self-check

1. A and B are independent, with $P(A) = 0.5$ and $P(B) = 0.2$. Find $P(A \cap B)$ and $P(A \cup B)$.
2. Deal two cards from a deck **without** putting the first back. Are “first card is a heart” and “second card is a heart” independent?
3. Can an event be independent of itself?

1.7 Interview question: odd dice

(A typical warm-up at Optiver, SIG or Jane Street. They want to see you test the definition, not trust your feeling of “connected”.)

Roll two fair dice. Let A = “the first die is odd”, B = “the second die is odd”, C = “the sum is odd”.

a. Are A and B independent?

b. Are A and C independent? (And B and C?)

c. Are A, B, C independent as a group of three?

d. For which totals s is the event “the sum is s” independent of “the first die shows 6”?

1.8 Solution

The room. 36 equally likely ordered pairs. Each die is odd with probability $3/6 = 1/2$.

(a) $A \cap B$: both odd, $3 \times 3 = 9$ worlds. $P(A \cap B) = 9/36 = 1/4 = \frac{1}{2} \times \frac{1}{2}$. **Independent.** (Two separate dice; nothing links them.)

(b) The sum is odd exactly when one die is odd and the other is even: $9 + 9 = 18$ worlds, so $P(C) = 1/2$. $A \cap C$ = first odd **and** second even: 9 worlds, $P = 1/4 = P(A)P(C)$. **Independent.** The same count works for B and C. In words: knowing the first die is odd tells you nothing about whether the sum is odd, because that still depends 50/50 on the second die.

(c) If A and B both happen, the sum is odd + odd = even, so C is **impossible**. $P(A \cap B \cap C) = 0$, but $P(A)P(B)P(C) = 1/8$. **Not independent as a group.** Each pair is independent, yet any two of the events together decide the third. This is called **pairwise but not mutual** independence.

(d) "First die is 6" has probability $1/6$. For a total s from 7 to 12, exactly one world has a first 6 and total s, so $P(\text{both}) = 1/36$. For s from 2 to 6 it is 0. Independence needs $P(\text{sum} = s) \times \frac{1}{6} = \frac{1}{36}$, so $P(\text{sum} = s) = 1/6$, which means 6 worlds. Only $s = 7$ has 6 worlds. **Answer: $s = 7$ only.**

Interview trick. Test independence with the "does the share move?" question, by counting inside the small room. For (b): in the small room "first die odd" (18 worlds), how many have an odd sum? Half of them, 9. Same 50% as the whole room. Done.

Sanity check. I counted all 36 worlds by computer: $P(A \cap B) = P(A \cap C) = P(B \cap C) = 1/4$, $P(A \cap B \cap C) = 0$, and among $s = 2, \ldots, 12$ only $s = 7$ gives $1/36 = P(\text{sum} = s)/6$.

Self-check answers. (1) $0.5 \times 0.2 = 0.1$; then $0.5 + 0.2 - 0.1 = 0.6$ by the addition rule. (2) No. $P(\text{second heart}) = 1/4$, but given the first was a heart, it is $12/51 \approx 0.235$. With replacement they would be independent. (3) Only if $P(A) = P(A)^2$, that is $P(A) = 0$ or 1. An event that might or might not happen always tells you about itself.

---

2. Derivatives: Futures, a Forward With a Referee

> **Where we are.** Lesson 3: long gains when the price rises, short gains when it falls; a pip in EURUSD is 0.0001. Lesson 4: pip value = pip × notional. Lesson 5: a forward fixes today an exchange rate for a later date, with payoff $N(S_T - K)$ for the buyer. Lesson 6: its fair price comes from interest rates (covered interest parity). Today: the same deal, traded on an exchange.

2.1 The scene

Recall the Hong Kong importer who bought EUR 5 million forward from her bank. That works because the bank knows her: it has checked her accounts and given her a credit line. But a forward is a **private promise**. If one side goes bust before the date, the other side may never be paid. This is counterparty risk (Lesson 5).

Now meet a second player: a small macro fund in Singapore that wants the **same** bet, long EUR 5 million against USD for three months. No big bank will give it a large credit line. So it goes to an exchange (CME in Chicago) and buys **EUR/USD futures**. The seller on the other side might be a US exporter hedging euro income, or another fund betting on a weaker euro. The two never meet and do not need to trust each other.

How can strangers make a three-month promise safely? Two inventions: a **referee** who stands in the middle, and a rule that **losses are paid every evening**.

2.2 The central picture: the bar tab vs pay-per-round

Think of a group night out.

- **Forward = the bar tab.** You drink all night and settle once at the end. If your friend's tab grows huge and he slips out the back door, you are left paying. Trust is everything.
- **Futures = pay-per-round, with a deposit.** The barman (the **clearing house**) asks each person for a deposit at the door. After every round, whoever is behind pays the barman, and the barman pays whoever is ahead. Nobody's debt can grow for long. If someone cannot pay a round, the barman closes their tab and keeps their deposit.

In the picture	In the maths and market
The barman in the middle	The **clearing house** (CME Clearing): buyer to every seller, seller to every buyer
Deposit at the door	**Initial margin**
Lowest deposit you may sit on	**Maintenance margin**
Settling up after each round	**Daily mark-to-market** (variation margin)
"Top up your deposit or leave"	**Margin call**
A standard drink from a menu	Standard contract: fixed size and dates

Where the analogy stops: in a bar, the barman takes the drinks bill. Here the clearing house takes **no** market risk itself. Every dollar one side loses goes to the other side. The clearing house only passes the money through and holds the deposits.

2.3 The mechanics, one at a time

Standard contracts. A forward can be any size and any date. A CME EUR/USD future is always **EUR 125,000**, quoted in dollars per euro, with fixed quarterly delivery months (March, June, September, December). One pip (0.0001) on one contract is worth $0.0001 \times 125{,}000 = \text{USD } 12.50$. Standard sizes are what allow strangers to trade the same thing all day.

The clearing house (novation). When the fund buys from the exporter, the clearing house steps in: it becomes the seller to the fund and the buyer to the exporter. Each side now only needs to trust the clearing house, which is backed by margin from everyone and

by its own default fund.

Margin. To open the position you post **initial margin**, a few percent of the notional. If losses push your account below the **maintenance margin**, you get a margin call and must top it back up to the **initial** level, usually by the next morning. Margin is a security deposit, not a price: you get it back when you close.

Daily mark-to-market. Each afternoon the exchange sets a **settlement price**. Your position is revalued to it, and the day's gain or loss is paid in cash, today. Then tomorrow starts fresh from that price.

2.4 Worked trade: the fund's EUR 5 million

The fund buys **40 contracts** ($40 \times 125{,}000 = \text{EUR } 5{,}000{,}000$) at **1.1700**.

- Pip value: $40 \times 12.50 = \text{USD } 500$ per pip.
- Margin (illustrative; the real figures change with volatility): initial USD 2,750 per contract, maintenance USD 2,500. For 40 contracts: initial **USD 110,000**, maintenance **USD 100,000**.

So the fund controls about USD 5.85 million of euros ($5{,}000{,}000 \times 1.17$) with USD 110,000, under 2%. That is leverage (Lesson 3).

The fund's cushion before a call is $110{,}000 - 100{,}000 = 10{,}000$, which is $10{,}000/500 = 20$ pips. So a settlement below 1.1680 on day one would trigger a call.

Day	Settlement	Move (pips)	Cash today (USD)	Account before top-up	Action	Account after
0	1.1700 (bought)			110,000	posts initial margin	110,000
1	1.1690	-10	-5,000	105,000	above 100,000: fine	105,000
2	1.1660	-30	-15,000	90,000	**margin call**: pays 20,000	110,000
3	1.1675	+15	+7,500	117,500	fine	117,500
4	1.1720	+45	+22,500	140,000	fine	140,000
5	1.1735 (sells)	+15	+7,500	147,500	closes, takes it all back	0

Step by step for day 2: the price fell 30 pips, $30 \times 500 = 15{,}000$. The account drops from 105,000 to 90,000, which is below 100,000. The fund must bring it back to the **initial** 110,000, so it pays in $110{,}000 - 90{,}000 = 20{,}000$.

Total profit. Add the daily cash: $-5{,}000 - 15{,}000 + 7{,}500 + 22{,}500 + 7{,}500 = 17{,}500$. Check with one line: $(1.1735 - 1.1700) \times 5{,}000{,}000 = 0.0035 \times 5{,}000{,}000 = 17{,}500$. And with the money: the fund put in $110{,}000 + 20{,}000 = 130{,}000$ and took out $147{,}500$, a gain of 17,500. All three agree.

Same deal as a forward? A forward at 1.1700, closed out at 1.1735, also makes 17,500 (ignoring the small discounting from Lesson 2), but in **one** payment at the end. The **total** is the same; the **timing** is different. With futures the fund had to find USD 20,000 of real cash on day 2, at the worst moment, even though the trade ended up a winner.

2.5 Why the timing can kill you

Metallgesellschaft, 1993. A US unit of this German group sold customers fixed-price oil for up to ten years ahead (long-dated forward promises), and hedged by buying short-dated oil futures. When oil prices fell, the hedge lost money **every day in cash**, while the gains on the customer contracts would only arrive over the years. The parent company cut the position under the pressure of margin calls, and losses were more than a billion dollars. The hedge was roughly right on paper; the cash flow timing was not survivable.

More recently, in March 2022 nickel on the London Metal Exchange more than doubled within a day, and margin calls on short positions were so large that the LME suspended trading and cancelled trades. Daily settlement protects the system, but it moves the pain to whoever is short of cash.

2.6 Futures price vs forward price

For short-dated FX, the futures price is almost exactly the forward price from Lesson 6. The tiny difference comes from daily settlement: gains received early can earn interest, and losses paid early cost interest. If those cash flows tend to come when interest rates are high or low, futures and forwards drift slightly apart. This matters for long-dated interest rate futures (a “convexity adjustment”, a later lesson), and hardly at all for a 3-month EURUSD contract.

2.7 Likely interview questions

Q1. You are long 10 EUR/USD futures at 1.1700 (EUR 125,000 each), initial margin USD 2,750 and maintenance USD 2,500 per contract. Where is your first margin call, and how much do you pay if the day settles at 1.1660? Pip value $10 \times 12.50 = 125$. Cushion $27{,}500 - 25{,}000 = 2{,}500$, which is 20 pips, so any settlement below 1.1680. At 1.1660 you lose 40 pips, $5{,}000$, leaving 22,500. You top up to 27,500: pay **USD 5,000**.

Q2. Who is your counterparty on a futures trade? The clearing house, through novation. It is buyer to every seller and seller to every buyer, and it is protected by margin, daily settlement and a default fund.

Q3. Forward or futures: which would a corporate hedger prefer, and why? Usually the forward: any size and date to match the invoice, and no daily cash calls, at the cost of needing a credit line with the bank. Futures suit players who want low counterparty risk, easy exit and transparent prices, and who can handle daily cash moves.

---

3. Algorithms & Data Structures: Two Pointers and Three Trades That Net to Zero

> **Where we are.** Lesson 1: Big-O, why n^2 beats n^3. Lesson 3: scanning an array in place. Lesson 4: hash sets. Lesson 6: sorting with `std::sort` in $O(n \log n)$. Today we use a **sorted** array to search for pairs much faster, by walking two indices towards each other.

3.1 The central picture: two people at the ends of a shelf

Picture a shelf of boxes sorted by weight, lightest on the left, heaviest on the right. You need two boxes that weigh exactly 10 kg together. Two helpers stand at the two ends of the shelf, each holding one box.

- Together they weigh **too much**? The only useful move is for the right-hand helper to step left, to a lighter box. Moving the left helper right would make it heavier still.
- **Too little**? The left helper steps right, to a heavier box.
- **Exactly right?** Found it.

Each step, one helper moves, and they never walk backwards. After at most n steps they meet. That is the **two pointers** technique.

In the picture	In the code
The shelf, sorted by weight	A sorted `vector<int>`
Left helper	Index `lo`, starts at 0
Right helper	Index `hi`, starts at $n - 1$
“Too heavy, right helper steps left”	`sum > target`: `--hi`
“Too light, left helper steps right”	`sum < target`: `++lo`
Helpers meet	`lo == hi`: no pair left

Small example. Shelf $[1, 3, 4, 6, 8, 11]$, target 10.

lo, hi	Pair	Sum	Move
0, 5	1 + 11	12	too big: `--hi`
0, 4	1 + 8	9	too small: `++lo`
1, 4	3 + 8	11	too big: `--hi`
1, 3	3 + 6	9	too small: `++lo`
2, 3	4 + 6	10	found

Five checks instead of the $\binom{6}{2} = 15$ pairs of the brute force.

Why is it safe to throw a box away? This is the key, so slowly. Say $1 + 11 = 12$ is too big. We drop the 11 forever. Is that safe? The 1 is the **lightest** box still in play. If even the lightest box plus 11 is too heavy, then **every** box plus 11 is too heavy. So 11 cannot be in any answer. In the same way, if the smallest sum involving the heaviest box is too small, the lightest box cannot be in any answer. Every step removes one box that provably cannot help. That is why we never miss a pair.

Where the analogy stops: the shelf **must be sorted**. On an unsorted shelf, "step left" does not mean "lighter", and the whole argument falls apart.

3.2 The problem

A desk has a list of trade P&Ls (whole numbers, in thousands). Find all **unique** groups of three trades whose P&Ls add up to exactly zero. (This is the classic "3Sum" interview problem.)

- Input: `vector<int> nums`, $3 \le n \le 3{,}000$, $-10^5 \le$ `nums[i]` $\le 10^5$.
- Output: every distinct triple of values $[a, b, c]$ with $a + b + c = 0$, using three different positions. "Distinct" is about values: $[-1, 0, 1]$ appears once even if there are many ways to pick it.

Input	Output
`[-1, 0, 1, 2, -1, -4]`	`[[-1,-1,2], [-1,0,1]]`
`[0, 1, 1]`	`[]`
`[0, 0, 0, 0]`	`[[0,0,0]]`
`[-5, -2, -2, 1, 3, 4, 4, 7]`	`[[-5,-2,7], [-5,1,4], [-2,-2,4]]`

3.3 Brute force first

Try every triple of positions $i < j < k$, keep those that sum to zero, sort each triple and put it in a `std::set` to remove repeats. There are $\binom{n}{3} \approx n^3/6$ triples. For $n = 3{,}000$ that is about 4.5×10^9: far too slow, but a perfect **checker** for testing.

A better idea uses a hash set (Lesson 4): for each pair, look up whether $-(a+b)$ exists. That is $O(n^2)$, but removing duplicate triples cleanly is fiddly and it uses $O(n)$ extra memory.

3.4 The insight: fix one, then two pointers

Sort the array. Now pick the **smallest** member of the triple, `nums[i]`. We need two later numbers adding up to `-nums[i]`. That is exactly the shelf problem on the part of the array to the right of `i`. So:

1. Sort `nums`.
2. For each anchor `i`: run two pointers with `lo = i + 1`, `hi = n - 1`, looking for `nums[lo] + nums[hi] = -nums[i]`.

Duplicates, the tricky part. Sorting puts equal values next to each other, which makes repeats easy to skip:

- **Anchor:** if `nums[i] == nums[i-1]`, this anchor value was already fully searched, so skip it.
- **After a hit:** move `lo` past every copy of its value and `hi` past every copy of its value. Otherwise $[-1, 0, 1]$ would be recorded again with the next 0 or 1.

An early stop. If the anchor `nums[i] > 0`, all later numbers are positive too, and three positives cannot sum to zero. Break.

Walk through example 1. Sorted: $[-4, -1, -1, 0, 1, 2]$.

- Anchor -4 (need 4 from the rest): $-1+2=1$, $-1+2=1$, $0+2=2$, $1+2=3$: always too small, `lo` walks to the end. Nothing.
- Anchor -1 (position 1, need 1): $-1+2=1$: **hit** $[-1, -1, 2]$. Skip copies: `lo` to the 0, `hi` to the 1. $0+1=1$: **hit** $[-1, 0, 1]$. Pointers cross.
- Anchor -1 (position 2): same value as before, skip.
- Anchor 0 (need 0): $1+2=3$, too big, `hi` steps down onto `lo`. Done.

Complexity. Sorting is $O(n \log n)$. There are up to n anchors, and each runs a two-pointer walk of at most n steps: $O(n^2)$ in total, which dominates. For $n = 3{,}000$ that is about 4.5 million steps (roughly $n^2/2$), instantly fast. Extra memory: $O(1)$ beyond the output and our sorted copy. Can we beat n^2? For general input, no known algorithm does fundamentally better, and an interviewer expects $O(n^2)$.

3.5 C++ solution

```cpp
// Daily Quant Digest 2026-11-02
// FR Lesson 7: two pointers. Problem: three trades that net to zero (3Sum).
// Compile: g++ -std=c++17 -O2 -Wall 2026-11-02.cpp -o three_sum && ./three_sum

#include <algorithm>
#include <array>
#include <cassert>
#include <iostream>
```

```cpp
#include <random>
#include <set>
#include <vector>

using std::array;
using std::vector;

// Warm-up: does a SORTED array contain two entries (at different positions)
// that add up to target? Two pointers walking towards each other.
bool hasPairWithSum(const vector<int>& sorted, int target) {
    int lo = 0;
    int hi = static_cast<int>(sorted.size()) - 1;
    while (lo < hi) {
        int sum = sorted[lo] + sorted[hi];
        if (sum == target) return true;
        if (sum < target) ++lo;   // need more: the small end steps up
        else --hi;                // need less: the big end steps down
    }
    return false;
}

// Main solution: all unique triplets {a, b, c} with a + b + c == 0.
// Sort, fix the smallest member, then two pointers on the rest.
// O(n^2) time, O(1) extra space besides the output (we sort our own copy).
vector<array<int, 3>> threeSum(vector<int> nums) {       // by value: we may reorder it
    std::sort(nums.begin(), nums.end());
    const int n = static_cast<int>(nums.size());
    vector<array<int, 3>> result;

    for (int i = 0; i + 2 < n; ++i) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;   // same anchor value: skip
        if (nums[i] > 0) break;                          // smallest is positive: no more zeros

        int lo = i + 1;
        int hi = n - 1;
        while (lo < hi) {
            int sum = nums[i] + nums[lo] + nums[hi];
            if (sum < 0) {
                ++lo;
            } else if (sum > 0) {
                --hi;
            } else {
                result.push_back({nums[i], nums[lo], nums[hi]});
                int leftVal = nums[lo];
                int rightVal = nums[hi];
                while (lo < hi && nums[lo] == leftVal) ++lo;    // jump past copies
                while (lo < hi && nums[hi] == rightVal) --hi;
            }
        }
    }
    return result;
}

// Brute force for testing: try every triple of positions, O(n^3).
vector<array<int, 3>> threeSumBrute(const vector<int>& nums) {
    std::set<array<int, 3>> found;                       // set removes duplicates and sorts
    const int n = static_cast<int>(nums.size());
    for (int i = 0; i < n; ++i)
        for (int j = i + 1; j < n; ++j)
            for (int k = j + 1; k < n; ++k)
                if (nums[i] + nums[j] + nums[k] == 0) {
                    array<int, 3> t = {nums[i], nums[j], nums[k]};
                    std::sort(t.begin(), t.end());
                    found.insert(t);
                }
    return vector<array<int, 3>>(found.begin(), found.end());
```

```
}

void print(const vector<array<int, 3>>& triples) {
    std::cout << "[";
    for (std::size_t t = 0; t < triples.size(); ++t) {
        const auto& [a, b, c] = triples[t];                 // structured binding
        std::cout << (t ? ", " : "") << "[" << a << "," << b << "," << c << "]";
    }
    std::cout << "]\n";
}

int main() {
    // Examples from the digest.
    vector<int> ex1 = {-1, 0, 1, 2, -1, -4};
    vector<int> ex2 = {0, 1, 1};
    vector<int> ex3 = {0, 0, 0, 0};
    vector<int> ex4 = {-5, -2, -2, 1, 3, 4, 4, 7};
    print(threeSum(ex1));   // [[-1,-1,2], [-1,0,1]]
    print(threeSum(ex2));   // []
    print(threeSum(ex3));   // [[0,0,0]]
    print(threeSum(ex4));   // [[-5,-2,7], [-5,1,4], [-2,-2,4]]

    vector<int> shelf = {1, 3, 4, 6, 8, 11};
    std::cout << std::boolalpha << hasPairWithSum(shelf, 10) << " "
              << hasPairWithSum(shelf, 13) << "\n";            // true false

    // Randomized cross-check against brute force. The fast version already
    // returns triples in sorted order (anchor ascending, then lo ascending).
    std::mt19937 rng(20261102);
    for (int trial = 0; trial < 3000; ++trial) {
        int n = std::uniform_int_distribution<int>(0, 25)(rng);
        int range = std::uniform_int_distribution<int>(1, 12)(rng);
        std::uniform_int_distribution<int> val(-range, range);
        vector<int> a(n);
        for (int& x : a) x = val(rng);
        auto fast = threeSum(a);
        auto slow = threeSumBrute(a);
        assert(fast == slow);
    }
    std::cout << "3000 random tests passed\n";
    return 0;
}
```

Output:

```
[[-1,-1,2], [-1,0,1]]
[]
[[0,0,0]]
[[-5,-2,7], [-5,1,4], [-2,-2,4]]
true false
3000 random tests passed
```

3.6 Walkthrough of the C++

`vector<int> threeSum(vector<int> nums)`: taking the input by value. We need to sort, which rearranges the data. Taking a copy means the caller's vector is left untouched. Compare with `hasPairWithSum(const vector<int>& sorted, ...)`, which only reads, so a `const` reference (Lesson 2) avoids copying.

`std::array<int, 3>`. A fixed-size array that knows its size and behaves like a value: you can copy it, compare two of them with `==` and `<`, and store it in a `vector` or a `set`. A `vector<int>` of size 3 would also work, but each one would allocate memory on the heap; `array` lives inline and is cheaper. Because `array` has a built-in `<` (it compares element by element, like words in a dictionary), `std::set<array<int,3>>` just works in the brute force.

`result.push_back({nums[i], nums[lo], nums[hi]});`. The braces build an `array<int,3>` on the spot. This is called brace (or list) initialization.

`static_cast<int>(nums.size())`. `size()` returns an **unsigned** type (`size_t`). If you write `nums.size() - 1` on an empty vector, it does not give -1; it wraps round to a huge positive number. Converting to `int` once, at the top, avoids this classic bug. It is also why the loop condition is `i + 2 < n` rather than `i < n - 2` on unsigned values.

`continue` and `break`. `continue` skips the rest of this loop round and goes to the next `i` (duplicate anchor). `break` leaves the loop completely (positive anchor: nothing more to find).

The duplicate-skipping loops. `while (lo < hi && nums[lo] == leftVal) ++lo;` We save the value first in `leftVal`, then step while we still see it. The `lo < hi` check comes first: `&&` stops as soon as the left side is false, so we never compare past the meeting point.

Integer overflow? Values are at most 10^5 in size, so a sum of three is at most 3×10^5, far inside `int` (about 2.1×10^9). With values up to 10^9 you would switch the sum to `long long` (Lesson 5).

`const auto& [a, b, c] = triples[t];`. A **structured binding** (C++17): it unpacks the three elements of the array into names `a`, `b`, `c`. `const auto&` means "a read-only reference, let the compiler work out the type", so nothing is copied.

`std::boolalpha`. Makes `cout` print `true` / `false` instead of `1` / `0`.

The random test. `std::mt19937` is the standard Mersenne Twister random generator, seeded with a fixed number so every run is the same. `uniform_int_distribution<int>(a, b)` turns its raw output into integers from a to b inclusive. We make 3,000 small random arrays with many repeated values (small ranges force duplicates, the risky case), and `assert` that the fast answer equals the brute-force answer. Both lists come out in the same sorted order: the brute force because a `set` is sorted, the fast version because anchors go up and, for a fixed anchor, `lo` only goes up. `assert` stops the program loudly if the condition is ever false.

Two other shapes of two pointers. Today's pointers walk **towards** each other. Another common shape has both pointers walking in the **same** direction, a slow "writer" and a fast "reader", for in-place jobs like removing duplicates from a sorted array. Tomorrow's lesson, the sliding window, grows out of that second shape.

Self-check. (1) Why must the array be sorted for the two-pointer pair search? (2) In `[-2, -2, 4]`, what stops the anchor loop from reporting it twice? (3) What is the complexity if you used two pointers inside two nested anchor loops (4Sum)?

Answers. (1) Because "move left" must mean "smaller"; that is what lets us discard a number for good. (2) The second -2 as an anchor is skipped by `nums[i] == nums[i-1]`. (3) $O(n^3)$: two loops for the anchors, times $O(n)$ for the walk.

---

4. Music: Cream, "Sunshine of Your Love" (1967)

Context

Cream were a trio formed in London in 1966: **Eric Clapton** on guitar, **Jack Bruce** on bass and vocals, and **Ginger Baker** on drums. Each was already a star player, so they are often called the first "supergroup". "Sunshine of Your Love" is on their second album, **Disraeli Gears** (November 1967), recorded at Atlantic Studios in New York with producer Felix Pappalardi and engineer Tom Dowd. The song is credited to Bruce, Clapton and the poet **Pete Brown**, who wrote the words. Released as a single in the US in 1968, it reached number 5 on the Billboard Hot 100, the band's biggest hit there.

The story behind it

The famous riff came from Jack Bruce. He came up with it after seeing Jimi Hendrix play in London, early in 1967, and he worked it out on his double bass at home. Pete Brown had been up all night with him trying to write, and the words about the coming dawn grew out of looking at the sky as the sun came up after a night of work. Clapton then added the chorus section. Cream played live more like a jazz group than a pop band, with long improvised solos, and this song became a centrepiece of their shows.

Why people love it

It is one of the riffs every guitarist learns early: a handful of notes from the blues scale, heavy and slow, played by guitar and bass together, so it hits like one huge instrument. It sits right at the turning point where blues rock was becoming hard rock, and you can hear the roots of the heavier bands that followed.

What to listen for

- **The riff in unison.** Guitar and bass play the same line together. Listen to how it leaves space between phrases instead of filling every beat.
- **Ginger Baker's drums.** Instead of a normal rock backbeat (snare hits on 2 and 4), he leans on the toms with a rolling, tribal pattern that pushes the first and third beats. It gives the song its slow, swaying weight.

- **Clapton’s “woman tone”.** Clapton’s name for the thick, rounded, almost vocal sound he got by turning the guitar’s tone control right down and pushing a loud amplifier. It is all over the solo.
- **The solo’s opening.** It starts by quoting the melody of the old standard “Blue Moon” before taking off. Then notice how the band lifts back into the riff at the end.
- **Two singers.** Bruce and Clapton share the vocals: Bruce’s higher, stronger voice and Clapton’s softer reply.